User-Level Resource-Aware Runtime System

1 Problem Statement

Modern computing platforms increasingly operate under dynamic and constrained resource conditions, such as fluctuating CPU availability, memory pressure, and contention from co-located workloads. Despite this, most applications rely entirely on operating system abstractions that hide resource dynamics and provide limited control over runtime behavior.

The goal of this project is to design and implement a *user-level runtime system* that explicitly monitors system resources and adapts application execution accordingly. The runtime must operate entirely in user space, without any modification to the operating system kernel.

The runtime should provide applications with:

- Visibility into system resource conditions
- Control over task scheduling and execution
- The ability to adapt behavior dynamically under resource stress

The emphasis of this project is on *systems design and implementation*, including clean abstractions, correctness, modularity, and experimental validation.

2 Project Scope

This is a multi-phase systems project intended to be completed over an extended period. The final system is expected to resemble a small but complete runtime environment rather than a single-purpose prototype.

The project scope includes:

- A task abstraction implemented entirely in user space
- Cooperative scheduling without kernel-level preemption
- Modular resource monitoring (CPU and memory at minimum)
- Adaptive scheduling decisions driven by runtime observations
- Quantitative evaluation under controlled workloads

The system should be extensible and structured so that new scheduling policies or resource signals can be added without major refactoring.

3 System Requirements

3.1 Task Abstraction and Runtime API

The runtime must expose a clear API to applications for:

- Creating and destroying tasks
- Voluntarily yielding execution
- Assigning task metadata (e.g., priority or resource hints)

Tasks may be implemented using user-level threads, coroutines, or explicit state machines. Reliance on OS-level thread scheduling for correctness is not allowed.

3.2 Scheduling Subsystem

The runtime must support multiple scheduling policies, including:

- A simple baseline policy (e.g., FIFO or round-robin)
- A priority-based policy
- A resource-aware adaptive policy

The scheduling policy must be changeable at runtime without restarting the application.

3.3 Resource Monitoring

The runtime must continuously monitor system resources using user-space mechanisms. At a minimum:

- CPU utilization must be sampled periodically
- Memory usage must be tracked at the process level

Monitoring overhead must be measured and reported as part of the evaluation.

3.4 Adaptive Behavior

The runtime must demonstrate adaptive behavior, such as:

- Throttling or deferring tasks under high CPU load
- Reducing task concurrency under memory pressure
- Switching scheduling policies based on resource thresholds

All adaptation decisions must be observable and logged.

4 Runtime Activity Dashboard

To demonstrate that the runtime is active and influencing execution, a simple runtime activity dashboard or console-based monitor must be provided.

At a minimum, the dashboard should display:

- Current CPU and memory utilization as observed by the runtime
- Number of active and runnable tasks
- Currently active scheduling policy
- Recent adaptation events

An indicative textual output:

```
[User-Level Runtime Active]
CPU Utilization     : 74%
Memory Usage        : 1.6 GB
Active Tasks        : 8
Scheduling Policy   : Resource-Aware
Adaptation Events   : Deferred Task T5 due to CPU pressure
```

The goal is clarity and observability, not visualization sophistication.

5 Workloads and Stress Testing

The project must include one or more workloads designed to stress the runtime:

- CPU-intensive workloads
- Memory-intensive workloads

- Mixed workloads exhibiting phase changes

At least one workload must demonstrate poor or undesirable behavior under a non-adaptive scheduler and improved behavior under the adaptive runtime.

Workloads must be reproducible and configurable.

6 Evaluation

The evaluation report must include:

- Comparison of different scheduling policies under identical workloads
- Evidence of adaptation and its impact on execution
- Discussion of monitoring and scheduling overhead
- Limitations and design trade-offs

Graphs and tables should be used where appropriate, accompanied by careful analysis.

7 Checkpoint-Based Submission Plan

The project is divided into incremental checkpoints, each increasing in difficulty and system complexity. Submissions are required every **20 days**.

Checkpoint 1 (Day 20): Runtime Skeleton

- Build system and project structure
- Basic task abstraction with cooperative yield
- Single-threaded execution loop

Checkpoint 2 (Day 40): Baseline Scheduling

- FIFO or round-robin scheduler
- Multiple runnable tasks
- Simple demo workload
- CPU and memory monitoring in user space
- Periodic sampling infrastructure
- Console output showing live resource data

Checkpoint 3 (Day 70): Adaptive Scheduling

- At least one adaptive scheduling policy
- Runtime-triggered adaptation decisions
- Logged adaptation events
- CPU- and memory-stressing workloads
- Demonstration of behavioral differences
- Preliminary evaluation results

Checkpoint 4 (Day 80): Final System and Evaluation

- Fully integrated runtime
- Complete evaluation report
- Clean codebase and documentation
- Final demo

Each checkpoint must build cleanly on the previous one; partial or throwaway implementations are strongly discouraged.

8 Deliverables

The final submission must include:

- Complete source code (C/C++)
- Design documentation
- Evaluation report
- Demo workloads
- README with build and execution instructions

All work must be original and reproducible on a standard Linux system.